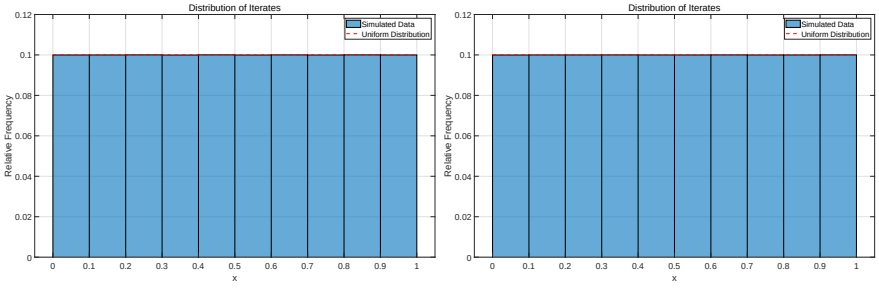
`Assignment 8` Iteration and Fractals

Shuifan Wang `shepherd.wang@foxmail.com`

May 15, 2025

Problem 1

$f(x) = F(x + \log_{10} 3)$ is the iterative function, where $F(x)$ is a function that directly takes the fractional part. Let $x_0 = 0$ and compute $x_n = f(x_{n-1})$, the distribution is demonstrated as follows. And it is also the case for π. We can see that they are uniform distributed, or more precisely, in equidistribution.



(a) Distribution w.r.t $\log_{10} 3$ (b) Distribution w.r.t π

Figure 1: Problem 1

```
%----------Problem 1----------
clear; clc; close all;

%% log10(3)
% Initialize Parameters
alpha = log10(3);
N = 100000;
x0 = 0;
```

```
x = zeros(N, 1);

% Iteration
x(1) = mod(x0 + alpha, 1);
for n = 2:N
    x(n) = mod(x(n-1) + alpha, 1); % Mod 1 operation
end

% Plot normalized histogram
figure;
histogram(x, 'BinEdges', 0:0.1:1, 'Normalization', 'probability');
title('Distribution of Iterates');
xlabel('x');
ylabel('Relative Frequency');
grid on;
hold on;
plot([0, 1], [0.1, 0.1], 'r--', 'LineWidth', 2);
legend('Simulated Data', 'Uniform Distribution');

% Print and Save
set(gca, 'FontSize', 18);
set(gcf, 'Position', [100, 100, 1600, 900]);
exportgraphics(gcf, 'Problem1_log.pdf', 'ContentType', 'vector');

disp(x(10000));

%% pi
clear; clc;
% Initialize Parameters
alpha = pi;
N = 100000;
x0 = 0;
x = zeros(N, 1);

% Iteration
x(1) = mod(x0 + alpha, 1);
for n = 2:N
    x(n) = mod(x(n-1) + alpha, 1); % Mod 1 operation
end

% Plot normalized histogram
figure;
histogram(x, 'BinEdges', 0:0.1:1, 'Normalization', 'probability');
title('Distribution of Iterates');
xlabel('x');
ylabel('Relative Frequency');
grid on;
hold on;
plot([0, 1], [0.1, 0.1], 'r--', 'LineWidth', 2);
legend('Simulated Data', 'Uniform Distribution');

% Print and Save
set(gca, 'FontSize', 18);
set(gcf, 'Position', [100, 100, 1600, 900]);
exportgraphics(gcf, 'Problem1_pi.pdf', 'ContentType', 'vector');

% No Output of Text.
```

Problem 2

Generate a Tree Fractal Diagram.

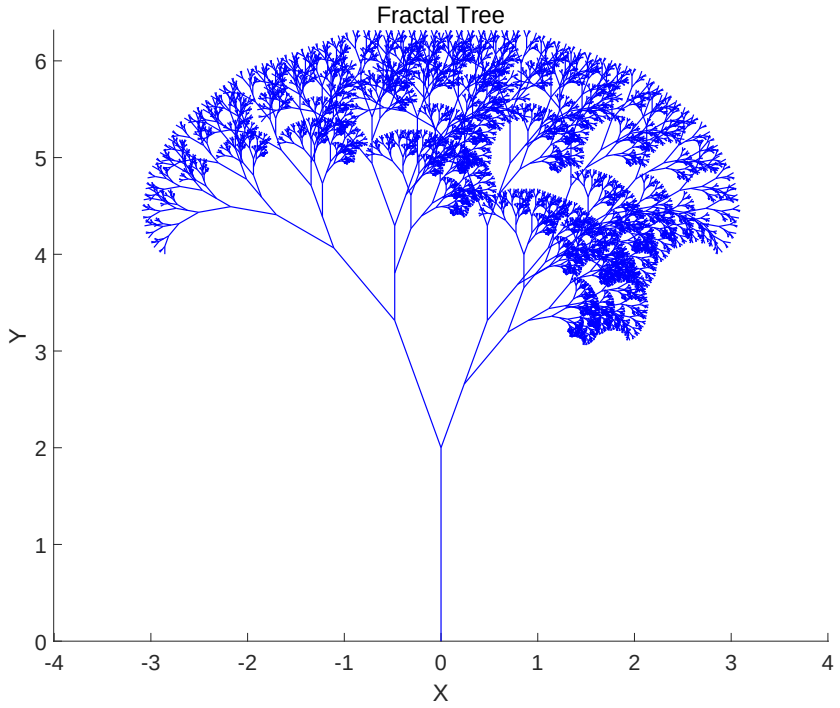


Figure 2: Fractal Tree

```
%----------Problem 2----------
clear; clc; close all;

%% Parameters Initialization
depth = 10;
initialLength = 2;
global theta1 theta2 Δ;
theta1 = 20;
theta2 = 40;
Δ = 0.7;

%% Initialize figure
figure;
hold on;
axis equal;
title('Fractal Tree');
xlabel('X');
ylabel('Y');
```

```
19
20 %% Main Recursive Function to Draw Branches
21 function drawBranch(startPoint, angle, length, depth)
22     global theta1 theta2 Δ;
23     if depth == 0
24         return
25     end
26
27     endPoint = startPoint + length * [cosd(angle), sind(angle)];
28     line([startPoint(1), endPoint(1)], ...
           [startPoint(2), endPoint(2)], 'Color', 'b', 'LineWidth', 1);
29
30     % Draw Left Branch
31     newAngle1 = angle + theta1;
32     newLength1 = length * Δ;
33     drawBranch(endPoint, newAngle1, newLength1, depth - 1);
34
35     % Draw Right Branch
36     newAngle2 = angle - theta1;
37     newLength2 = length * Δ;
38     drawBranch(endPoint, newAngle2, newLength2, depth - 1);
39
40     % Draw Small Branch
41     newLength3 = length * Δ*0.5;
42     newAngle3 = angle - theta2;
43     endPointR = endPoint + newLength3 * [cosd(newAngle2), ...
           sind(newAngle2)];
44     drawBranch(endPointR, newAngle3, newLength3, depth - 1);
45 end
46
47 %% Run and Export
48 drawBranch([0, 0], 90, initialLength, depth);
49 hold off;
50
51 set(gca, 'FontSize', 18);
52 set(gcf, 'Position', [100, 100, 1600, 900]);
53 exportgraphics(gcf, 'Problem2.pdf', 'ContentType', 'vector');
54
55 % No Output of Text.
```

Problem 3

We have scaling function $\varphi(x)$ and wavelet function $\psi(x)$ defined by this, where $h_k, 0 \leq k \leq 9$ and $g_k = (-1)^{k-1} h_{9-k}$ are given parameters.

$$\varphi(x) = \sum_{k=0}^{9} h_k \varphi(2x - k)$$
$$\psi(x) = \sum_{k=0}^{9} g_k \varphi(2x - k)$$

The support $\{x : \varphi(x) \neq 0\} = [0, 9]$ is the support.
We first compute $\varphi(x)$ at integer points. Let

$$A_{ij} = \begin{cases} h_{2i-j}, & \text{if } 2i - j \text{ is in the support,} \\ 0, & \text{else.} \end{cases}$$

Therefore we have

$$A\varphi = \varphi, \text{ where } A = \left[\varphi(1), \varphi(2), \ldots, \varphi(8)\right]^T.$$

After normalizing $\sum_{k=0}^{9} \varphi(k) = 1$, then we initialize other value as 0 and use cascade algorithm to compute the scaling function and then the wavelet function. Finally, we check

$$\int_0^9 \varphi(x)\psi(x)\mathrm{d}x = -0.00011213 \approx 0,$$

which is quite reasonable. The figure of these two functions are demonstrated below.

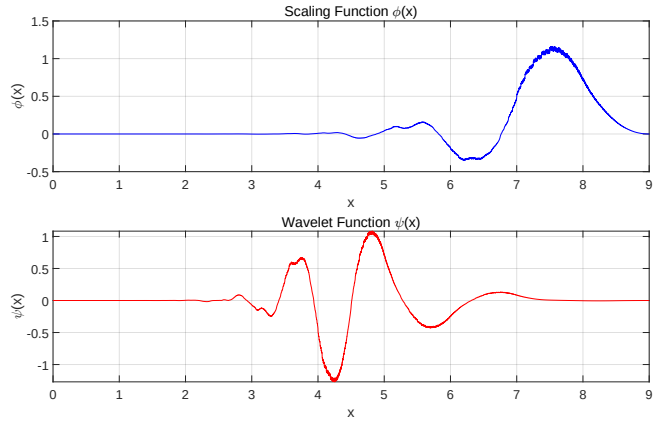


Figure 3: Problem 3

```
%----------Problem 3----------
clear; clc; close all;

%% Initialize Parameter Matrix
h = [0.00471427938400, -0.00179210101860, -0.008826800108660, ...
     0.109702658642161, -0.045601131884100, -0.342656715382664, ...
     0.195766961347502, 1.024326944260331, 0.853943542705429, ...
     0.226418982583462];
g = h(end:-1:1) .* (-1).^(0:9);
A = zeros(8, 8);
for i = 1:8
    for j = 1:8
        k = 2*i - j;
        if k ≥ 0 && k ≤ 9
            A(i, j) = h(k+1);
        end
    end
end

%% Compute the Eigenvector and Normalize
[V, D] = eig(A);
eigvals = diag(D);
[¬, idx] = min(abs(eigvals - 1));
phi_vec = V(:,idx);
phi_vec = phi_vec / sum(phi_vec);

%% Set up a Grid and Initialize phi(x) at Integer Points
J = 10; % refinement level
dx = 1 / 2^J;
x = 0:dx:9;
N = length(x);
phi = zeros(1, N);
for n = 0:9
    idx = round(n / dx) + 1;
    if n ≥ 1 && n ≤ 8
        phi(idx) = phi_vec(n);
    else
        phi(idx) = 0;
    end
end

%% Cascade algorithm to Approximate phi(x)
for iter = 1:10
    phi_new = zeros(1, N);
    for i = 1:N
        for l = 0:9
            k = round((2*x(i) - l) / dx) + 1;
            if k ≥ 1 && k ≤ N
                phi_new(i) = phi_new(i) + h(l+1) * phi(k);
            end
        end
    end
    phi = phi_new;
end

%% Compute psi(x)
psi = zeros(1, N);
```

```
for i = 1:N
    for l = 0:9
        k = round((2*x(i) - l) / dx) + 1;
        if k ≥ 1 && k ≤ N
            psi(i) = psi(i) + g(l+1) * phi(k);
        end
    end
end

%% Plot phi and psi
figure;
subplot(2, 1, 1);
plot(x, phi, 'b-');
title('Scaling Function \phi(x)');
xlabel('x');
ylabel('\phi(x)');
grid on;

subplot(2, 1, 2);
plot(x, psi, 'r-');
title('Wavelet Function \psi(x)');
xlabel('x');
ylabel('\psi(x)');
grid on;

%% Print and Export
integral_approx = sum(phi .* psi) * dx;
disp(['Approximate integral of phi*psi over [0,9]: ', ...
    num2str(integral_approx)]);
if abs(integral_approx) < 1e-3
    disp('Orthogonality condition is verified.');
else
    disp('Orthogonality condition is not verified.');
end

set(gcf, 'Position', [100, 100, 1600, 900]);
ax = findobj(gcf, 'Type', 'axes');
for i = 1:length(ax)
    set(ax(i), 'FontSize', 18);
end
exportgraphics(gcf, 'Problem3.pdf', 'ContentType', 'vector');

% Output:
% Approximate integral of phi*psi over [0,9]: -0.00011213
% Orthogonality condition is verified.
```